CO Assignment 1

Study and explanation using perf tool to observe different statistics in a program

Name: Arnav Singhal
MIS: 612203012
SY B. Tech. Comp. Div 1

About the Perf Tool:

The perf tool in Linux is a powerful performance analysis utility used to profile and monitor system and application performance. It provides detailed insights into CPU behaviour, memory usage, cache efficiency and other key performance metrics. By tracking both hardware events (such as CPU cycles, cache misses and branch predictions) and software events (like page faults and context switches), perf helps identify bottlenecks, optimize code and improve execution efficiency. It supports various commands like perf stat for high-level performance metrics, perf record for profiling, and perf list to view available events. This makes perf an essential tool for debugging performance issues and optimizing system workloads.

Installation:

```
arnav@arnav-desktop-1: ~
arnav@arnav-desktop-1:~$ sudo apt install linux-tools-$(uname -r) linux-tools-generic
[sudo] password for arnav:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfprint-2-tod1 libfwupdplugin1 libllvm9 libxmlb1 shim
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  linux-hwe-5.15-tools-5.15.0-125 linux-tools-5.4.0-208 linux-tools-5.4.0-208-generic linux-tools-common
The following NEW packages will be installed:
  linux-hwe-5.15-tools-5.15.0-125 linux-tools-5.15.0-125-generic linux-tools-5.4.0-208
  linux-tools-5.4.0-208-generic linux-tools-common linux-tools-generic
0 upgraded, 6 newly installed, 0 to remove and 132 not upgraded.
Need to get 13.1 MB of archives.
After this operation, 61.8 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 linux-tools-common all 5.4.0-208.228 [204 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 linux-hwe-5.15-tools-5.15.0-125 amd64 5.15.0-12
5.135~20.04.1 [7,321 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 linux-tools-5.15.0-125-generic amd64 5.15.0-125
.135~20.04.1 [2,016 B]
Get:4 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 linux-tools-5.4.0-208 amd64 5.4.0-208.228 [5,60
6 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 linux-tools-5.4.0-208-generic amd64 5.4.0-208.2
28 [1,996 B]
Get:6 http://in.archive.ubuntu.com/ubuntu focal-updates/main amd64 linux-tools-generic amd64 5.4.0.208.204 [2,464
B]
Fetched 13.1 MB in 4s (3,418 kB/s)
Selecting previously unselected package linux-tools-common.
(Reading database ... 186477 files and directories currently installed.)
```

```
B]
Fetched 13.1 MB in 4s (3,418 kB/s)
Selecting previously unselected package linux-tools-common.
(Reading database ... 186477 files and directories currently installed.)
Preparing to unpack .../0-linux-tools-common_5.4.0-208.228_all.deb ...
Unpacking linux-tools-common (5.4.0-208.228) ...
Selecting previously unselected package linux-hwe-5.15-tools-5.15.0-125.
Preparing to unpack .../1-linux-hwe-5.15-tools-5.15.0-125_5.15.0-125.135~20.04.1_amd64.deb ...
Unpacking linux-hwe-5.15-tools-5.15.0-125 (5.15.0-125.135~20.04.1) ...
Selecting previously unselected package linux-tools-5.15.0-125-generic.
Preparing to unpack .../2-linux-tools-5.15.0-125-generic_5.15.0-125.135~20.04.1_amd64.deb ...
Unpacking linux-tools-5.15.0-125-generic (5.15.0-125.135~20.04.1) ...
Selecting previously unselected package linux-tools-5.4.0-208.
Preparing to unpack .../3-linux-tools-5.4.0-208_5.4.0-208.228_amd64.deb ...
Unpacking linux-tools-5.4.0-208 (5.4.0-208.228) ...
Selecting previously unselected package linux-tools-5.4.0-208-generic.
Preparing to unpack .../4-linux-tools-5.4.0-208-generic_5.4.0-208.228_amd64.deb ...
Unpacking linux-tools-5.4.0-208-generic (5.4.0-208.228) ...
Selecting previously unselected package linux-tools-generic.
Preparing to unpack .../5-linux-tools-generic_5.4.0.208.204_amd64.deb ...
Unpacking linux-tools-generic (5.4.0.208.204) ...
Setting up linux-tools-common (5.4.0-208.228) ...
Setting up linux-hwe-5.15-tools-5.15.0-125 (5.15.0-125.135~20.04.1) ...
Setting up linux-tools-5.15.0-125-generic (5.15.0-125.135~20.04.1) ...
Setting up linux-tools-5.4.0-208 (5.4.0-208.228) ...
Setting up linux-tools-5.4.0-208-generic (5.4.0-208.228) ...
Setting up linux-tools-generic (5.4.0.208.204) ...
Processing triggers for man-db (2.9.1-1) ...
arnav@arnav-desktop-1:~$
```

List of commands:

```
arnav@arnav-desktop-1:~$ sudo sysctl -w kernel.perf_event_paranoid=1
kernel.perf_event_paranoid = 1
arnav@arnav-desktop-1:~$
arnav@arnav-desktop-1:~$ perf --help

 usage: perf [--version] [--help] [OPTIONS] COMMAND [ARGS]

 The most commonly used perf commands are:
   annotate        Read perf.data (created by perf record) and display annotated code
   archive         Create archive with object files with build-ids found in perf.data file
   bench           General framework for benchmark suites
   buildid-cache   Manage build-id cache.
   buildid-list    List the buildids in a perf.data file
   c2c             Shared Data C2C/HITM Analyzer.
   config          Get and set variables in a configuration file.
   daemon          Run record sessions on background
   data            Data file related processing
   diff            Read perf.data files and display the differential profile
   evlist          List the event names in a perf.data file
   ftrace          simple wrapper for kernel's ftrace functionality
   inject          Filter to augment the events stream with additional information
   iostat          Show I/O performance metrics
   kallsyms        Searches running kernel for symbols
   kmem            Tool to trace/measure kernel memory properties
   kvm             Tool to trace/measure kvm guest os
   list            List all symbolic event types
   lock            Analyze lock events
   mem             Profile memory accesses
   record          Run a command and record its profile into perf.data
   report          Read perf.data (created by perf record) and display the profile
   sched           Tool to trace/measure scheduler properties (latencies)
```

```
arnav@arnav-desktop-1: ~
  buildid-list    List the buildids in a perf.data file
  c2c             Shared Data C2C/HITM Analyzer.
  config          Get and set variables in a configuration file.
  daemon          Run record sessions on background
  data            Data file related processing
  diff            Read perf.data files and display the differential profile
  evlist          List the event names in a perf.data file
  ftrace          simple wrapper for kernel's ftrace functionality
  inject          Filter to augment the events stream with additional information
  iostat          Show I/O performance metrics
  kallsyms        Searches running kernel for symbols
  kmem            Tool to trace/measure kernel memory properties
  kvm             Tool to trace/measure kvm guest os
  list            List all symbolic event types
  lock            Analyze lock events
  mem             Profile memory accesses
  record          Run a command and record its profile into perf.data
  report          Read perf.data (created by perf record) and display the profile
  sched           Tool to trace/measure scheduler properties (latencies)
  script          Read perf.data (created by perf record) and display trace output
  stat            Run a command and gather performance counter statistics
  test            Runs sanity tests.
  timechart       Tool to visualize total system behavior during a workload
  top             System profiling tool.
  version         display the version of perf binary
  probe           Define new dynamic tracepoints
  trace           strace inspired tool

 See 'perf help COMMAND' for more information on a specific command.

arnav@arnav-desktop-1:~$
```

List of supported events:

```
arnav@arnav-desktop-1: ~
arnav@arnav-desktop-1:~$ perf list

List of pre-defined events (to be used in -e):

  duration_time                                      [Tool event]

  branch-instructions OR cpu/branch-instructions/    [Kernel PMU event]
  branch-misses OR cpu/branch-misses/                [Kernel PMU event]
  bus-cycles OR cpu/bus-cycles/                      [Kernel PMU event]
  cache-misses OR cpu/cache-misses/                  [Kernel PMU event]
  cache-references OR cpu/cache-references/          [Kernel PMU event]
  cpu-cycles OR cpu/cpu-cycles/                      [Kernel PMU event]
  instructions OR cpu/instructions/                  [Kernel PMU event]
  mem-loads OR cpu/mem-loads/                        [Kernel PMU event]
  mem-stores OR cpu/mem-stores/                      [Kernel PMU event]
  ref-cycles OR cpu/ref-cycles/                      [Kernel PMU event]
  topdown-fetch-bubbles OR cpu/topdown-fetch-bubbles/ [Kernel PMU event]
  topdown-recovery-bubbles OR cpu/topdown-recovery-bubbles/ [Kernel PMU event]
  topdown-slots-issued OR cpu/topdown-slots-issued/  [Kernel PMU event]
  topdown-slots-retired OR cpu/topdown-slots-retired/ [Kernel PMU event]
  topdown-total-slots OR cpu/topdown-total-slots/    [Kernel PMU event]
  cstate_core/c3-residency/                          [Kernel PMU event]
:
```

EVENTS:

The perf tool supports a list of measurable events. The tool and underlying kernel interface can measure events coming from different sources. On each processor, those events get mapped onto actual events provided by the CPU, if they exist, otherwise the event cannot be used.

Types of Events:

Hardware Events:
Hardware events refer to events that are generated by the hardware components of the system, such as the CPU, memory subsystem and other hardware units. These events provide insights into the behaviour and performance of the hardware itself, which can be extremely valuable for analyzing and optimizing system performance.

Software Events:
Software events refer to specific types of performance events or counters that can be measured and analyzed by perf without relying on specialized hardware performance counters. Software events are generated by the software itself, typically through the use of various instrumentation and tracing techniques. These events provide insights into the behaviour and performance of your software, applications or the Linux kernel.

Some Common Events:

1. **cpu-cycles**

   perf stat -e cpu-cycles ./prog

   CPU cycles, often referred to simply as "cycles," are a fundamental concept in computer architecture and performance measurement. They represent the basic unit of time in which a central processing unit (CPU) processes instructions and performs its operations. CPU cycles are a measure of how quickly the CPU can execute instructions and are closely related to the clock speed of the CPU. The speed of a computer processor, or CPU, is determined by the Clock Cycle, which is the amount of time between two pulses of an oscillator. Generally speaking, the higher number of pulses per second, the faster the computer processor will be able to process information. The clock speed is measured in Hz, typically either megahertz (MHz) or gigahertz (GHz). For example, a 4GHz processor performs 4,000,000,000 clock cycles per second. Computer processors can execute one or more instructions per clock cycle, depending on the type of

processor. Early computer processors and slower processors can only execute one instruction per clock cycle, but faster, more advanced processors can execute multiple instructions per clock cycle, processing data more efficiently.

2. **instructions**

   perf stat -e instructions ./prog

   Computer instructions are a set of machine language instructions that a particular processor understands and executes. A computer performs tasks on the basis of the instruction provided. An instruction comprises of groups called fields. These fields include: The Operation code (Opcode) field which specifies the operation to be performed. The Address field which contains the location of the operand, i.e., register or memory location. The Mode field which specifies how the operand will be located.
   MODE OPCODE OPERAND/ADDRESS
   A computer's instructions can be any length and have any number of addresses. The arrangement of a computer's registers determines the different address fields in the instruction format. The instruction can be classified as three, two, and one address instruction or zero address instruction, depending on the number of address fields.

3. **page-faults**

   perf stat -e page-faults ./prog

   The page-faults event in perf is used to measure the number of page faults encountered during the execution of a program. Page faults occur when a program attempts to access a memory page that is not currently in physical RAM and must be loaded from secondary storage, such as swap space or a disk.

4. **major-faults**

   perf stat -e major-faults ./prog

   The major-faults event in perf is used to measure the number of major page faults encountered during the execution of a program. Major page faults occur when a program attempts to access a memory page that is not currently in physical RAM and must be loaded from secondary storage, such as swap space or a disk.

5. **minor-faults**

   perf stat -e minor-faults ./prog

   The minor-faults event in perf is used to measure the number of minor page faults encountered during the execution of a program. Minor page faults occur when a program attempts to access a memory page that is not currently in physical RAM but is already available in other forms of fast-access memory, such as swap space or other parts of the virtual memory system.

6. **cpu-clock**

   perf stat -e cpu-clock ./prog

   The cpu-clock event in perf is used to measure the time the CPU spends executing instructions for a specific program. It provides information about the total CPU time consumed during the execution of the program.

7. **task-clock**

   perf stat -e task-clock ./prog

   The task-clock event in perf is used to measure the wall-clock time consumed by a specific task (process or thread). It represents the total time that the task has been running, including both the time it has spent executing instructions and the time it has been waiting (e.g., due to sleep or I/O operations).

Other Software Events:

1. **alignment-faults**

   perf stat -e alignment-faults ./prog

   An alignment fault is a type of software event that you can measure to understand how often alignment faults occur in your program. Alignment faults happen when data or instructions are accessed at addresses that do not adhere to the required alignment constraints.

2. **bpf-output**

   sudo perf stat -e bpf-output ./prog

   The bpf-output event in perf is used to measure the number of output events generated by BPF (Berkeley Packet Filter) programs. BPF is a framework in the Linux kernel that allows you to write and run custom packet filtering and processing programs.

3. **context-switches**

   sudo perf stat -e context-switches ./prog

   Context switches are a critical aspect of operating system and multitasking management. They involve the process of saving the state of one process or thread and restoring the state of another, allowing multiple processes or threads to share the CPU.

4. **cpu-migrations**

   sudo perf-stat cpu-migrations ./prog

   The cpu-migrations event in perf is used to monitor the number of times a process or thread migrates (moves) from one CPU core to another. CPU migrations occur when the Linux scheduler decides to move a process or thread to a different CPU core.

5. **dummy**

   perf stat -e dummy ./prog

   In perf a "dummy event" is not a real performance event but rather a placeholder or a way to activate certain functionality without measuring a specific hardware event.

6. **emulation-faults**

   perf stat -e emulation-faults ./prog

   The emulation-faults event in perf is used to measure the number of emulation faults encountered during the execution of a program. Emulation faults occur when a program attempts to execute instructions that are not natively supported by the CPU architecture and must be emulated or translated by the system.

Sample Program - Matrix Addition:

1. **Naïve Approach**

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 1024 //Matrix size

int** create_matrix() {
    int **matrix = (int**)malloc(SIZE * sizeof(int *));
    for(int i = 0; i < SIZE; i++)
        matrix[i] = (int*)malloc(SIZE * sizeof(int));
    return matrix;
}

void init_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        for(int j = 0; j < SIZE; j++)
            matrix[i][j] = rand() % 100;
}

void add_matrices(int **A, int **B, int **C) {
    for(int i = 0; i < SIZE; i++) {
        for(int j = 0; j < SIZE; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }
}

void free_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        free(matrix[i]);
    free(matrix);
}

int main() {
    int **A = create_matrix();
    int **B = create_matrix();
    int **C = create_matrix();
```

```
    init_matrix(A);
    init_matrix(B);

    add_matrices(A, B, C);

    printf("Matrix addition completed.\n");

    free_matrix(A);
    free_matrix(B);
    free_matrix(C);

    return 0;
}
```

Output and Performance:

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ gcc -O2 -Wall matrix_add.c -o a
arnav@arnav-desktop-1:~/Desktop/CO$ ./a
Matrix addition completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e cycles,instructions,cache-misses,page-faults -r 5 ./a
Matrix addition completed.
Matrix addition completed.
Matrix addition completed.
Matrix addition completed.
Matrix addition completed.

 Performance counter stats for './a' (5 runs):

       8,28,60,887      cycles                                                  ( +-  0.11% )
      17,98,48,249      instructions              #    2.17  insn per cycle     ( +-  0.02% )
          7,68,910      cache-misses                                            ( +-  1.90% )
             3,138      page-faults                                             ( +-  0.02% )

          0.025411 +- 0.000836 seconds time elapsed  ( +-  3.29% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

2. **Optimized Approach (Using loop unrolling)**

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 1024 //Matrix size
#define UNROLL_FACTOR 4 //Loop unrolling factor

int** create_matrix() {
    int **matrix = (int**)malloc(SIZE * sizeof(int *));
    for(int i = 0; i < SIZE; i++)
        matrix[i] = (int*)malloc(SIZE * sizeof(int));
    return matrix;
}

void init_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        for(int j = 0; j < SIZE; j++)
            matrix[i][j] = rand() % 100;
}

void add_matrices(int **A, int **B, int **C) {
    for(int i = 0; i < SIZE; i++) {
        for(int j = 0; j < SIZE; j += UNROLL_FACTOR) {
            C[i][j] = A[i][j] + B[i][j];
            C[i][j + 1] = A[i][j + 1] + B[i][j + 1];
            C[i][j + 2] = A[i][j + 2] + B[i][j + 2];
            C[i][j + 3] = A[i][j + 3] + B[i][j + 3];
        }
    }
}

void free_matrix(int **matrix) {
    for(int i = 0; i < SIZE; i++)
        free(matrix[i]);
    free(matrix);
}

int main() {
    int **A = create_matrix();
```

```
    int **B = create_matrix();
    int **C = create_matrix();

    init_matrix(A);
    init_matrix(B);

    add_matrices(A, B, C);

    printf("Matrix addition completed.\n");

    free_matrix(A);
    free_matrix(B);
    free_matrix(C);

    return 0;
}
```

Output and Performance:

```
arnav@arnav-desktop-1: ~/Desktop/CO
arnav@arnav-desktop-1:~/Desktop/CO$ gcc -O2 -Wall matrix_add_optimized.c -o a
arnav@arnav-desktop-1:~/Desktop/CO$ ./a
Matrix addition completed.
arnav@arnav-desktop-1:~/Desktop/CO$ perf stat -e cycles,instructions,cache-misses,page-faults -r 5 ./a
Matrix addition completed.
Matrix addition completed.
Matrix addition completed.
Matrix addition completed.
Matrix addition completed.

 Performance counter stats for './a' (5 runs):

          9,97,752      cycles                                                  ( +-  3.06% )
          8,07,586      instructions              #    0.80  insn per cycle     ( +-  0.63% )
             1,126      cache-misses                                            ( +-234.32% )
                51      page-faults                                             ( +-  1.00% )

         0.0005745 +- 0.0000232 seconds time elapsed  ( +-  4.03% )

arnav@arnav-desktop-1:~/Desktop/CO$
```

CONCLUSION:

1. This study examined the use of perf for performance analysis, focusing on its ability to track system metrics like clock cycles, instructions, cache-misses, page faults and execution time.
2. Events were categorized into hardware and software events, providing a structured approach to understanding system performance.
3. Performance was analyzed for matrix addition, which was initially performed using the naïve approach and later using the optimized approach. Loop unrolling was used to improve the code and this led to a significant improvement in performance.
4. The insights gained from this study serve as a foundation for future performance tuning and optimization.